

Azure DevOps / TFS MCP Server Guide for OpenCode

Complete Setup for Azure DevOps and Team Foundation Server Integration

Important Note: As of May 2026, there is **no official Azure DevOps/TFS MCP server** from Microsoft or the Model Context Protocol community. This guide covers:

- 1. **Alternative solutions** that work today
- 2. **How to create a custom MCP server** for Azure DevOps
- 3. **Workarounds** using existing tools

Table of Contents

- 1. [Understanding TFS vs Azure DevOps](#)
- 2. [Current State of MCP Support](#)
- 3. [Alternative Solutions](#)
- 4. [Creating a Custom MCP Server](#)
- 5. [Using Azure DevOps CLI with OpenCode](#)
- 6. [REST API Integration](#)
- 7. [Feature Comparison: What You'd Want](#)
- 8. [Roadmap: Building Your Own MCP](#)
- 9. [Community Alternatives](#)
- 10. [FAQ](#)

Understanding TFS vs Azure DevOps

What's the Difference?

Feature	TFS (Team Foundation Server)	Azure DevOps
Type	On-premises server	Cloud service (also available on-premises)
Current Status	Legacy, no longer developed	Active, modern
Latest Version	TFS 2018 (final version)	Continuously updated
Rebranding	-	Formerly VSTS (Visual Studio Team Services)
Recommendation	⚠️ Migrate to Azure DevOps	✅ Modern choice

Azure DevOps Services

Azure DevOps provides:

- **Azure Repos:** Git repositories and TFVC
- **Azure Pipelines:** CI/CD
- **Azure Boards:** Work item tracking (like Jira)
- **Azure Test Plans:** Testing tools
- **Azure Artifacts:** Package management

Migration Path

TFS 2018 → Azure DevOps Server (on-premises) → Azure DevOps Services (cloud)
↓
(Recommended upgrade path)

For this guide: We'll focus on **Azure DevOps** (both cloud and server) as it's the current platform. Most solutions work for both TFS 2018 and Azure DevOps with minimal changes.

Current State of MCP Support

Official MCP Servers (May 2026)

✅ Available:

- GitHub MCP (official)
- GitLab MCP (community)
- Atlassian MCP (official - Jira & Confluence)

❌ NOT Available:

- Azure DevOps MCP (no official server)
- TFS MCP (no official server)
- Azure Repos MCP (no standalone server)

Why No Official Support?

1. **Microsoft focus:** Microsoft is investing in GitHub Copilot, not MCP for Azure DevOps
2. **Market share:** GitHub has larger developer mindshare
3. **Community size:** Smaller MCP community for Azure DevOps
4. **Complexity:** Azure DevOps has many services (Repos, Boards, Pipelines, etc.)

Good News

Azure DevOps has **excellent REST APIs** that can be used to build:

- Custom MCP servers
 - Direct integrations
 - CLI-based workflows
-

Alternative Solutions

Solution 1: Use Azure DevOps CLI with OpenCode (Easiest)

What is it?

Azure DevOps CLI (`az devops`) is an official command-line tool that OpenCode can use via the bash tool.

Installation:

```
# Install Azure CLI first
# Windows
winget install -e --id Microsoft.AzureCLI
```

```
# macOS
brew install azure-cli

# Linux
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

# Then install Azure DevOps extension
az extension add --name azure-devops
```

Configuration:

```
# Login to Azure DevOps
az login

# Set default organization
az devops configure --defaults organization=https://dev.azure.com/yourorg
project=YourProject

# Create Personal Access Token (PAT)
# Go to: https://dev.azure.com/yourorg/_usersSettings/tokens
# Store PAT:
export AZURE_DEVOPS_EXT_PAT=your_pat_token_here
```

Usage with OpenCode:

```
You: "List open work items in Azure DevOps"
OpenCode: *Uses bash tool to run: az boards work-item query*

You: "Create a work item for this bug"
OpenCode: *Uses bash tool to run: az boards work-item create*
```

Pros:

-  Works immediately
-  Official Microsoft tool
-  No custom MCP server needed
-  Full Azure DevOps functionality

Cons:

-  Not as seamless as native MCP
-  Requires bash tool calls
-  Less context-aware
-  More verbose

Solution 2: Create Custom MCP Server (Advanced)

What is it?

Build your own MCP server using Azure DevOps REST APIs.

When to use:

- You want native MCP integration

- You need specific workflows
- You're comfortable with Node.js/Python
- You want the best OpenCode experience

See: [Creating a Custom MCP Server](#) section below

Solution 3: Use REST API Directly (Medium Complexity)

What is it?

OpenCode can use bash tool to make curl requests to Azure DevOps REST APIs.

Example:

```
# Get work items
curl -u :$AZURE_DEVOPS_PAT \
  https://dev.azure.com/yourorg/YourProject/_apis/wit/wiql?api-version=7.0 \
  -H "Content-Type: application/json" \
  -d '{"query": "SELECT [System.Id] FROM WorkItems WHERE [System.State] = '\''Active'\''"}'
```

Pros:

-  Direct API access
-  Full control
-  No additional tools needed

Cons:

-  More complex
-  Requires API knowledge
-  More error-prone

Creating a Custom MCP Server

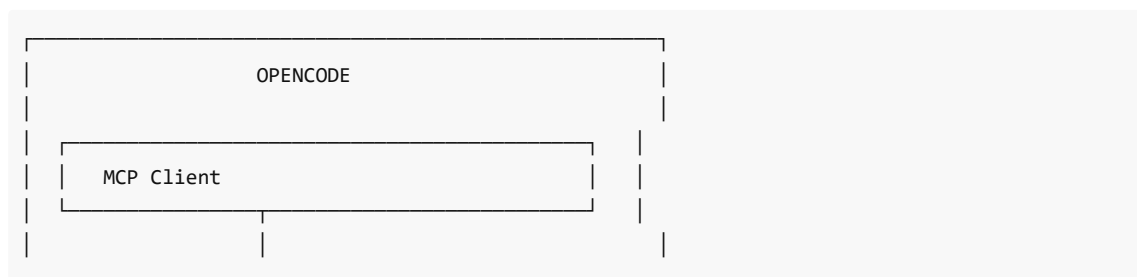
Overview

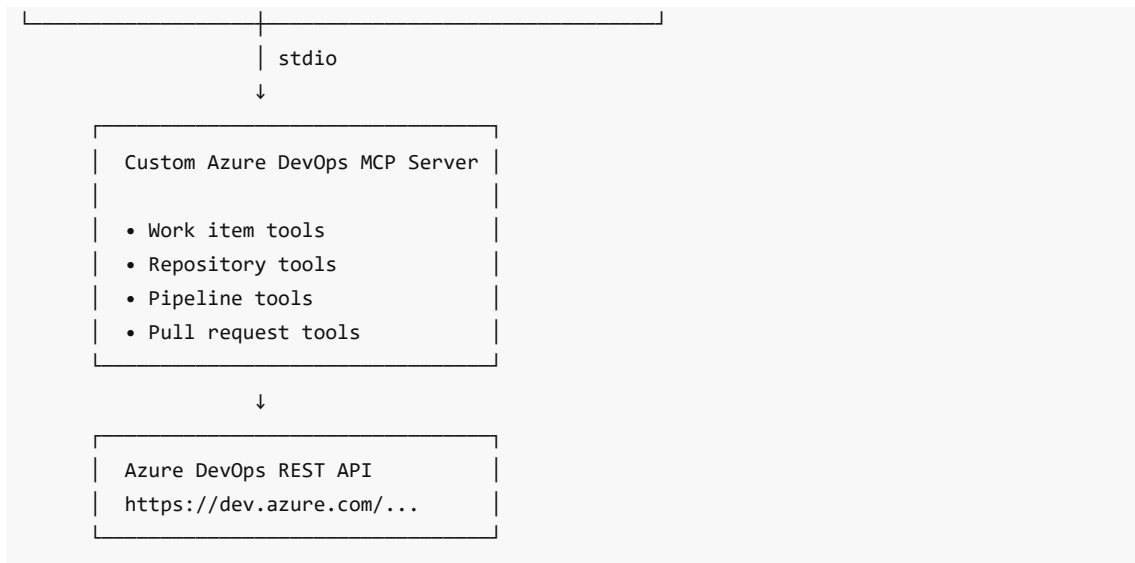
You can create a custom MCP server that wraps Azure DevOps APIs. Here's how:

Prerequisites

- Node.js 18+ or Python 3.10+
- Azure DevOps account
- Personal Access Token (PAT)
- MCP SDK knowledge

Architecture





Step 1: Set Up MCP Server Project

Option A: TypeScript/Node.js:

```
# Create project
mkdir mcp-server-azuredevops
cd mcp-server-azuredevops

# Initialize
npm init -y

# Install dependencies
npm install @modelcontextprotocol/sdk
npm install azure-devops-node-api
npm install @types/node --save-dev

# Install TypeScript
npm install typescript --save-dev
npx tsc --init
```

Option B: Python:

```
# Create project
mkdir mcp-server-azuredevops
cd mcp-server-azuredevops

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install mcp
pip install azure-devops
```

Step 2: Implement MCP Server (TypeScript Example)

src/index.ts :

```
#!/usr/bin/env node
import { Server } from '@modelcontextprotocol/sdk/server/index.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import {
  CallToolRequestSchema,
  ListToolsRequestSchema,
} from '@modelcontextprotocol/sdk/types.js';
import * as azdev from 'azure-devops-node-api';

// Configuration from environment variables
const orgUrl = process.env.AZURE_DEVOPS_ORG_URL || '';
const token = process.env.AZURE_DEVOPS_PAT || '';

if (!orgUrl || !token) {
  console.error('Missing required environment variables:');
  console.error(' AZURE_DEVOPS_ORG_URL - Your organization URL');
  console.error(' AZURE_DEVOPS_PAT - Personal Access Token');
  process.exit(1);
}

// Create Azure DevOps connection
const authHandler = azdev.getPersonalAccessTokenHandler(token);
const connection = new azdev.WebApi(orgUrl, authHandler);

// Create MCP server
const server = new Server(
  {
    name: 'azuredevops-mcp-server',
    version: '1.0.0',
  },
  {
    capabilities: {
      tools: {},
    },
  }
);

// Define available tools
server.setRequestHandler(ListToolsRequestSchema, async () => {
  return {
    tools: [
      {
        name: 'azuredevops_list_work_items',
        description: 'List work items using WIQL query',
        inputSchema: {
          type: 'object',
          properties: {
            project: {

```

```

        type: 'string',
        description: 'Project name',
    },
    query: {
        type: 'string',
        description: 'WIQL query (e.g., "SELECT [System.Id] FROM WorkItems WHERE [System.State] = \'Active\')',
    },
    required: ['project', 'query'],
},
{
    name: 'azuredevops_get_work_item',
    description: 'Get work item details by ID',
    inputSchema: {
        type: 'object',
        properties: {
            id: {
                type: 'number',
                description: 'Work item ID',
            },
        },
        required: ['id'],
    },
},
{
    name: 'azuredevops_create_work_item',
    description: 'Create a new work item',
    inputSchema: {
        type: 'object',
        properties: {
            project: {
                type: 'string',
                description: 'Project name',
            },
            type: {
                type: 'string',
                description: 'Work item type (Bug, Task, User Story, etc.)',
            },
            title: {
                type: 'string',
                description: 'Work item title',
            },
            description: {
                type: 'string',
                description: 'Work item description',
            },
        },
        required: ['project', 'type', 'title'],
    },
},
},

```

```

{
  name: 'azuredevops_list_repositories',
  description: 'List Git repositories in a project',
  inputSchema: {
    type: 'object',
    properties: {
      project: {
        type: 'string',
        description: 'Project name',
      },
    },
    required: ['project'],
  },
},
{
  name: 'azuredevops_list_pull_requests',
  description: 'List pull requests in a repository',
  inputSchema: {
    type: 'object',
    properties: {
      project: {
        type: 'string',
        description: 'Project name',
      },
      repository: {
        type: 'string',
        description: 'Repository name or ID',
      },
      status: {
        type: 'string',
        description: 'PR status: active, completed, abandoned, all',
        enum: ['active', 'completed', 'abandoned', 'all'],
      },
    },
    required: ['project', 'repository'],
  },
},
],
};
});

// Handle tool calls
server.setRequestHandler(CallToolRequestSchema, async (request) => {
  try {
    const { name, arguments: args } = request.params;

    switch (name) {
      case 'azuredevops_list_work_items': {
        const witApi = await connection.getWorkItemTrackingApi();
        const wiql = {
          query: args.query,
        };
      }
    }
  }
});

```

```

const result = await witApi.queryByWiql(wiql, args.project);

if (!result.workItems || result.workItems.length === 0) {
  return {
    content: [
      {
        type: 'text',
        text: 'No work items found',
      },
    ],
  };
}

// Get full work item details
const ids = result.workItems.map((wi) => wi.id!);
const workItems = await witApi.getWorkItems(ids);

return {
  content: [
    {
      type: 'text',
      text: JSON.stringify(workItems, null, 2),
    },
  ],
};
}

case 'azuredevops_get_work_item': {
  const witApi = await connection.getWorkItemTrackingApi();
  const workItem = await witApi.getWorkItem(args.id);

  return {
    content: [
      {
        type: 'text',
        text: JSON.stringify(workItem, null, 2),
      },
    ],
  };
}

case 'azuredevops_create_work_item': {
  const witApi = await connection.getWorkItemTrackingApi();

  const document = [
    {
      op: 'add',
      path: '/fields/System.Title',
      value: args.title,
    },
  ];
}

```

```

    if (args.description) {
      document.push({
        op: 'add',
        path: '/fields/System.Description',
        value: args.description,
      });
    }
  }

  const workItem = await witApi.createWorkItem(
    null,
    document,
    args.project,
    args.type
  );

  return {
    content: [
      {
        type: 'text',
        text: `Created work item ${workItem.id}: ${workItem.fields['System.Title']}`,
      },
    ],
  };
}

case 'azuredevops_list_repositories': {
  const gitApi = await connection.getGitApi();
  const repos = await gitApi.getRepositories(args.project);

  return {
    content: [
      {
        type: 'text',
        text: JSON.stringify(repos, null, 2),
      },
    ],
  };
}

case 'azuredevops_list_pull_requests': {
  const gitApi = await connection.getGitApi();
  const searchCriteria = {
    status: args.status || 'active',
  };
  const prs = await gitApi.getPullRequests(
    args.repository,
    searchCriteria,
    args.project
  );

  return {
    content: [

```

```

        {
            type: 'text',
            text: JSON.stringify(prs, null, 2),
        },
    ],
};
}

default:
    return {
        content: [
            {
                type: 'text',
                text: `Unknown tool: ${name}`,
            },
        ],
        isError: true,
    };
}
} catch (error: any) {
    return {
        content: [
            {
                type: 'text',
                text: `Error: ${error.message}`,
            },
        ],
        isError: true,
    };
}
});

// Start server
async function main() {
    const transport = new StdioServerTransport();
    await server.connect(transport);
    console.error('Azure DevOps MCP server running on stdio');
}

main().catch(console.error);

```

package.json :

```

{
  "name": "mcp-server-azuredevops",
  "version": "1.0.0",
  "description": "MCP server for Azure DevOps",
  "type": "module",
  "main": "dist/index.js",
  "bin": {
    "mcp-server-azuredevops": "./dist/index.js"
  }
}

```

```
  },
  "scripts": {
    "build": "tsc",
    "prepare": "npm run build"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "latest",
    "azure-devops-node-api": "^12.0.0"
  },
  "devDependencies": {
    "@types/node": "^20.0.0",
    "typescript": "^5.0.0"
  }
}
```

tsconfig.json :

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "moduleResolution": "Node16",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

Step 3: Build and Test

```
# Build TypeScript
npm run build

# Test locally
export AZURE_DEVOPS_ORG_URL="https://dev.azure.com/yourorg"
export AZURE_DEVOPS_PAT="your_pat_token"

# Run server (should start and wait for input)
node dist/index.js
```

Step 4: Install Locally

```
# Install globally
npm install -g .

# Or link for development
npm link
```

Step 5: Configure OpenCode

~/ .config/opencode/opencode.json :

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "azuredevops": {
      "type": "stdio",
      "command": "mcp-server-azuredevops",
      "env": {
        "AZURE_DEVOPS_ORG_URL": "{env:AZURE_DEVOPS_ORG_URL}",
        "AZURE_DEVOPS_PAT": "{env:AZURE_DEVOPS_PAT}"
      },
      "enabled": true
    }
  }
}
```

Set environment variables:

```
# Windows (PowerShell)
$env:AZURE_DEVOPS_ORG_URL = "https://dev.azure.com/yourorg"
$env:AZURE_DEVOPS_PAT = "your_pat_token"

# macOS/Linux
export AZURE_DEVOPS_ORG_URL="https://dev.azure.com/yourorg"
export AZURE_DEVOPS_PAT="your_pat_token"
```

Step 6: Test with OpenCode

```
opencode

# Try commands:
> "List active work items in MyProject"
> "Create a bug for login issue in MyProject"
> "Show me pull requests in my-repo"
```

Using Azure DevOps CLI with OpenCode

Complete Setup Guide

Step 1: Install Azure CLI

Windows:

```
winget install -e --id Microsoft.AzureCLI

# Or using installer
# Download from: https://aka.ms/installazurecliwindows
```

macOS:

```
brew install azure-cli
```

Linux:

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

Step 2: Install Azure DevOps Extension

```
az extension add --name azure-devops
```

Step 3: Authenticate

```
# Login to Azure
az login

# Or use PAT (Personal Access Token)
# Create PAT at: https://dev.azure.com/yourorg/_usersSettings/tokens
export AZURE_DEVOPS_EXT_PAT=your_pat_token_here

# Set defaults
az devops configure --defaults \
  organization=https://dev.azure.com/yourorg \
  project=YourProject
```

Step 4: Test Commands

```
# List work items
az boards query --wiql "SELECT [System.Id], [System.Title] FROM WorkItems WHERE
[System.State] = 'Active'"

# Create work item
az boards work-item create \
  --title "Login bug" \
  --type Bug \
  --description "Users cannot log in"
```

```
# List repositories
az repos list

# List pull requests
az repos pr list --status active
```

Step 5: Use with OpenCode

OpenCode can now use these commands via the bash tool:

```
You: "Show me active work items"
OpenCode: *Runs: az boards query --wiql "..."*

You: "Create a bug for the login issue"
OpenCode: *Runs: az boards work-item create ...*

You: "List my pull requests"
OpenCode: *Runs: az repos pr list --creator @me*
```

REST API Integration

Authentication

Create Personal Access Token (PAT):

1. Go to: https://dev.azure.com/yourorg/_usersSettings/tokens
2. Click "New Token"
3. Set scopes:
 - ☒ Work Items: Read, Write
 - ☒ Code: Read, Write
 - ☒ Build: Read
 - ☒ Release: Read
4. Copy token (starts with `ey...` or similar)

Store PAT:

```
# Environment variable
export AZURE_DEVOPS_PAT="your_pat_token"

# Or use in commands directly
PAT="your_pat_token"
```

Common API Endpoints

Work Items

Query work items:

```
curl -u :$AZURE_DEVOPS_PAT \
  https://dev.azure.com/yourorg/YourProject/_apis/wit/wiql?api-version=7.0 \
```

```
-H "Content-Type: application/json" \  
-d '{  
  "query": "SELECT [System.Id], [System.Title] FROM WorkItems WHERE [System.State] =  
'\''Active'\''"  
}'
```

Get work item:

```
curl -u :$AZURE_DEVOPS_PAT \  
https://dev.azure.com/yourorg/YourProject/_apis/wit/workitems/123?api-version=7.0
```

Create work item:

```
curl -u :$AZURE_DEVOPS_PAT \  
https://dev.azure.com/yourorg/YourProject/_apis/wit/workitems/$Bug?api-version=7.0 \  
-X POST \  
-H "Content-Type: application/json-patch+json" \  
-d '[  
  {  
    "op": "add",  
    "path": "/fields/System.Title",  
    "value": "Login bug"  
  },  
  {  
    "op": "add",  
    "path": "/fields/System.Description",  
    "value": "Users cannot log in"  
  }  
]
```

Git Repositories

List repositories:

```
curl -u :$AZURE_DEVOPS_PAT \  
https://dev.azure.com/yourorg/YourProject/_apis/git/repositories?api-version=7.0
```

List pull requests:

```
curl -u :$AZURE_DEVOPS_PAT \  
https://dev.azure.com/yourorg/YourProject/_apis/git/repositories/repo-id/pullrequests?api-version=7.0
```

Create pull request:

```
curl -u :$AZURE_DEVOPS_PAT \  
https://dev.azure.com/yourorg/YourProject/_apis/git/repositories/repo-id/pullrequests?api-version=7.0 \  
-X POST \  

```

```
-H "Content-Type: application/json" \  
-d '{  
  "sourceRefName": "refs/heads/feature-branch",  
  "targetRefName": "refs/heads/main",  
  "title": "Add new feature",  
  "description": "This PR adds a new feature"  
}'
```

Feature Comparison: What You'd Want

Ideal Azure DevOps MCP Features

If/when an official or community MCP server exists, these are the features you'd want:

Work Items (Azure Boards)

- ☒ Create work items (Bug, Task, User Story, Epic, Feature)
- ☒ Query work items (WIQL)
- ☒ Update work items (state, assignment, fields)
- ☒ Add comments to work items
- ☒ Link work items
- ☒ Attach files
- ☒ Get work item history
- ☒ Manage sprints and iterations

Repositories (Azure Repos)

- ☒ List repositories
- ☒ Get file contents
- ☒ Create/update files
- ☒ List branches
- ☒ Create branches
- ☒ Get commits
- ☒ Compare branches

Pull Requests

- ☒ List pull requests
- ☒ Create pull requests
- ☒ Get PR details
- ☒ Add PR comments
- ☒ Approve/reject PRs
- ☒ Complete (merge) PRs
- ☒ Get PR diff

Pipelines

- ☒ List pipelines
- ☒ Trigger pipeline runs
- ☒ Get pipeline run status
- ☒ Get pipeline logs
- ☒ List releases

Test Plans

- ☒ List test plans

- ☒ Get test results
 - ☒ Create test runs
-

Roadmap: Building Your Own MCP

Phase 1: Minimum Viable Product (1-2 weeks)

Core Features:

- ☐ Work item query (WIQL)
- ☐ Get work item details
- ☐ Create work items
- ☐ List repositories
- ☐ List pull requests

Tools:

- `azuredevops_query_work_items`
- `azuredevops_get_work_item`
- `azuredevops_create_work_item`
- `azuredevops_list_repositories`
- `azuredevops_list_pull_requests`

Phase 2: Enhanced Features (2-4 weeks)

Additional Features:

- ☐ Update work items
- ☐ Add work item comments
- ☐ Create pull requests
- ☐ Get PR details and comments
- ☐ List branches
- ☐ Get file contents

Phase 3: Advanced Features (4-8 weeks)

Additional Features:

- ☐ Pipeline triggers and status
- ☐ Work item linking
- ☐ File operations (create/update)
- ☐ PR reviews and completion
- ☐ Test plan integration

Phase 4: Polish & Distribution (2-4 weeks)

Tasks:

- ☐ Comprehensive error handling
- ☐ Rate limiting
- ☐ Caching
- ☐ Documentation

- ☐ Unit tests
- ☐ Publish to npm
- ☐ Submit to MCP registry

Community Alternatives

Option 1: Request Official Support

How:

1. Open issue on MCP GitHub: <https://github.com/modelcontextprotocol/servers/issues>
2. Explain use case and demand
3. Offer to help build it

Template:

Feature Request: Azure DevOps / TFS MCP Server

Problem

Azure DevOps/TFS is widely used in enterprises, but there's no MCP server support.

Proposed Solution

Create an official Azure DevOps MCP server with:

- Work item management
- Repository operations
- Pull request workflow
- Pipeline integration

Use Cases

- Create work items from code
- Link PRs to work items
- Query backlogs
- Automate workflows

Community Support

- [X] I would use this
- [X] I can help build this

Option 2: Check Community Servers

MCP Registry:

- Check: <https://github.com/modelcontextprotocol/servers>
- Search for: "azure", "devops", "tfs", "ado"

npm Search:

```
npm search mcp azure devops
npm search mcp tfs
```

Option 3: Hire/Commission Development

If you need this urgently:

- Hire a developer to build custom MCP server
 - Estimated effort: 40-80 hours (\$2,000-\$8,000)
 - Can be open-sourced for community
-

FAQ

Q: Can I use the GitHub MCP with Azure Repos?

A: No. Azure Repos is a different platform with different APIs. You need an Azure DevOps-specific integration.

Q: Does Azure DevOps CLI work well with OpenCode?

A: Yes! It's currently the best solution. OpenCode can use `az devops` commands via the bash tool. See [Using Azure DevOps CLI](#).

Q: Can I migrate from TFS to Azure DevOps?

A: Yes. Microsoft provides migration tools:

- TFS 2018 → Azure DevOps Server (on-premises)
 - Azure DevOps Server → Azure DevOps Services (cloud)
 - Guide: <https://learn.microsoft.com/azure/devops/migrate/>
-

Q: What permissions does my PAT need?

A: Minimum:

- Work Items: Read, Write
 - Code: Read, Write (if using repos/PRs)
 - Build: Read (if using pipelines)
-

Q: Can I use this with Azure DevOps Server (on-premises)?

A: Yes! Just change the URL:

```
export AZURE_DEVOPS_ORG_URL="https://your-server:8080/tfs/DefaultCollection"
```

Q: Is there a Python version of the custom MCP server?

A: Yes, you can build one using:

- `mcp` Python package
- `azure-devops` Python package

See the TypeScript example and adapt to Python.

Q: How do I create a Personal Access Token?

A:

1. Go to: `https://dev.azure.com/yourorg/_usersSettings/tokens`
2. Click "New Token"
3. Set expiration and scopes
4. Copy token immediately (you won't see it again)

Q: Can OpenCode help me build the custom MCP server?

A: Yes! OpenCode with Serena can help you:

- Read the MCP SDK documentation
- Write the TypeScript/Python code
- Debug issues
- Create tests

Q: Where can I find Azure DevOps API documentation?

A:

- REST API: <https://learn.microsoft.com/rest/api/azure/devops/>
- Node.js SDK: <https://github.com/microsoft/azure-devops-node-api>
- Python SDK: <https://github.com/microsoft/azure-devops-python-api>

Q: Can I contribute to making this real?

A: Yes! Options:

1. Build and open-source a custom MCP server
2. Request feature in MCP community
3. Share this guide with others who need it
4. Contribute to documentation

Summary

Current Best Options (May 2026)

Option	Ease	Power	When to Use
Azure DevOps CLI	★★★★★	★★★★★	Quick setup, official tool
Custom MCP Server	★★	★★★★★★	Best integration, requires dev work
Direct REST API	★★★	★★★★★★	Full control, more complex

Recommendation

For most users: Start with **Azure DevOps CLI** (`az devops`)

- ☒ Works immediately
- ☒ Official Microsoft support
- ☒ Full functionality
- ☒ No custom code needed

For best experience: Build **Custom MCP Server**

-  Native MCP integration
-  Seamless OpenCode experience
-  Can open-source for community
-  Requires development effort

Next Steps

1. **Immediate:** Install Azure DevOps CLI and try with OpenCode
2. **Short-term:** Evaluate if CLI meets your needs
3. **Long-term:** Consider building custom MCP if CLI isn't enough
4. **Community:** Request official support in MCP GitHub

Quick Start: Azure DevOps CLI Method

```
# 1. Install Azure CLI
winget install -e --id Microsoft.AzureCLI # Windows
brew install azure-cli # macOS

# 2. Install Azure DevOps extension
az extension add --name azure-devops

# 3. Login
az login

# 4. Create PAT
# Go to: https://dev.azure.com/yourorg/_usersSettings/tokens
export AZURE_DEVOPS_EXT_PAT="your_pat"

# 5. Configure defaults
az devops configure --defaults \
  organization=https://dev.azure.com/yourorg \
  project=YourProject

# 6. Test
az boards query --wiql "SELECT [System.Id] FROM WorkItems"

# 7. Use with OpenCode
opencode
> "List active work items in Azure DevOps"
```

Done! 🎉

Additional Resources

Official Documentation

- **Azure DevOps REST API:** <https://learn.microsoft.com/rest/api/azure/devops/>
- **Azure CLI:** <https://learn.microsoft.com/cli/azure/devops/>
- **Node.js SDK:** <https://github.com/microsoft/azure-devops-node-api>
- **Python SDK:** <https://github.com/microsoft/azure-devops-python-api>

MCP Resources

- **MCP Specification:** <https://modelcontextprotocol.io/>
- **MCP SDK (TypeScript):** <https://github.com/modelcontextprotocol/typescript-sdk>
- **MCP SDK (Python):** <https://github.com/modelcontextprotocol/python-sdk>
- **MCP Servers Registry:** <https://github.com/modelcontextprotocol/servers>

Community

- **OpenCode Discord:** <https://discord.gg/opencode>
- **Azure DevOps Community:** <https://developercommunity.visualstudio.com/>

Document Information:

- **Version:** 1.0
- **Last Updated:** May 15, 2026
- **Status:** No official MCP server available
- **Recommended Solution:** Azure DevOps CLI
- **Alternative:** Build custom MCP server

Note: This guide will be updated when official Azure DevOps MCP support becomes available. Check the MCP servers registry periodically for updates.